

# Deteccao de Anomalias e Fraudes em Transacoes de Cartao de Credito

Relatorio tecnico do sistema completo de machine learning, deep learning, API REST e dashboard interativo

|             |                                                                                                                                               |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Dataset     | 284.807 transacoes / 492 fraudes                                                                                                              |
| Modelos     | 10 abordagens supervisionadas e de anomalia                                                                                                   |
| Features    | Mais de 100 variaveis apos engenharia                                                                                                         |
| Interfaces  | CLI, FastAPI, Streamlit e PDF                                                                                                                 |
| Repositorio | <a href="https://github.com/davidsonsilva/desafio-dio-fraudes-cartao-credito">github.com/davidsonsilva/desafio-dio-fraudes-cartao-credito</a> |

Davidson Silva - 2026

## 1. Contexto e objetivo

O projeto utiliza o dataset publico Credit Card Fraud Detection, com 284.807 transacoes e somente 492 fraudes (aproximadamente 0,173%). O objetivo e construir um fluxo reproduzivel para detectar transacoes suspeitas, comparar diferentes familias de modelos e disponibilizar a inferencia por aplicacao web e API.

Como a classe positiva e extremamente rara, acuracia nao e usada como criterio principal. A avaliacao prioriza PR-AUC, recall, precisao, F1 e matriz de confusao.

## 2. Arquitetura da solucao

- Entrada pelo CSV real do Kaggle ou por dados sinteticos compativeis para smoke tests.
- Validacao do contrato com Time, Amount, Class e V1 ate V28.
- Divisao estratificada entre treino e teste antes das transformacoes aprendidas.
- Engenharia deterministica de mais de 100 features e escalonamento robusto.
- Treinamento e comparacao de dez modelos com estrategias proprias de balanceamento.
- Selecao do modelo por PR-AUC e calibracao dinamica do threshold.
- Persistencia de um artefato unico consumido pela CLI, FastAPI e Streamlit.
- Registro auditavel de feedback confirmado para o proximo retreinamento.

## 3. Engenharia de features

As variaveis originais sao enriquecidas com representacao ciclica do horario, indicadores temporais, transformacoes de Amount, interacoes entre variaveis PCA, modulos, quadrados e estatisticas agregadas. RobustScaler reduz a influencia de outliers sem aprender informacao do conjunto de teste.

## 4. Modelos avaliados

| Modelo               | Tipo           | Tratamento do desbalanceamento |
|----------------------|----------------|--------------------------------|
| Regressao Logistica  | Supervisionado | SMOTE e peso balanceado        |
| Random Forest        | Supervisionado | SMOTE e peso balanceado        |
| Extra Trees          | Supervisionado | SMOTE e peso balanceado        |
| HistGradientBoosting | Supervisionado | SMOTE                          |
| XGBoost              | Supervisionado | scale_pos_weight               |
| TensorFlow / Keras   | Deep learning  | class weights e early stopping |
| Isolation Forest     | Anomalia       | Somente exemplos normais       |
| Local Outlier Factor | Novidade       | novelty=True; somente normais  |
| One-Class SVM        | Novidade       | Somente exemplos normais       |
| PCA Reconstruction   | Anomalia       | Erro de reconstrucao           |

## 5. Avaliacao e threshold

Cada estimador gera um score contiuuo. A curva precision-recall e usada para localizar o threshold que maximiza F1 entre os pontos que atendem ao recall minimo configurado. Se a restricao nao puder ser atingida, o melhor F1

disponível e utilizado. O vencedor é selecionado por PR-AUC, com F1 como desempate.

## 6. Interfaces e artefatos

- FastAPI: health check, informações do modelo, previsão individual, lote e feedback.
- Streamlit: indicadores, upload de CSV, classificação, ranking de risco e exportação.
- CLI: treinamento com dataset real ou sintético e geração do relatório dinâmico.
- Jolliib: modelo, transformações, threshold, colunas, versão e métricas.
- ReportLab: relatórios PDF técnicos e de resultados.

## 7. Processo de execução

1. Criar ambiente Python 3.11 a 3.13 e instalar o pacote com `pip install -e "[dev]"`.
2. Baixar `creditcard.csv` pelo script Kaggle ou usar o modo `--demo`.
3. Executar `python -m fraud_detection.cli train`.
4. Iniciar o dashboard com `streamlit run app/streamlit_app.py`.
5. Iniciar a API com `uvicorn fraud_detection.api:app --reload`.
6. Gerar o relatório de métricas com `python -m fraud_detection.cli report`.

## 8. Qualidade, segurança e limitações

- Testes cobrem contrato dos dados, features, threshold, API e catálogo de modelos.
- Dataset, credenciais, modelos e feedback gerados não são versionados.
- Jolliib deve ser carregado somente de uma fonte confiável.
- Resultados sintéticos validam integração, não desempenho real.
- Produção requer validação temporal, monitoramento de drift e revisão humana.
- As variáveis PCA anonimizadas limitam explicações semânticas detalhadas.

## 9. Referencias oficiais

### **Dataset Kaggle**

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

### **scikit-learn**

<https://scikit-learn.org/stable/>

### **Novelty and Outlier Detection**

[https://scikit-learn.org/stable/modules/outlier\\_detection.html](https://scikit-learn.org/stable/modules/outlier_detection.html)

### **XGBoost**

<https://xgboost.readthedocs.io/en/stable/>

### **TensorFlow**

<https://www.tensorflow.org/?hl=pt-br>

### **TensorFlow - Imbalanced Data**

[https://www.tensorflow.org/tutorials/structured\\_data/imbalanced\\_data](https://www.tensorflow.org/tutorials/structured_data/imbalanced_data)

### **Streamlit**

<https://docs.streamlit.io/>

### **FastAPI**

<https://fastapi.tiangolo.com/>

### **imbalanced-learn SMOTE**

[https://imbalanced-learn.org/stable/references/generated/imblearn.over\\_sampling.SMOTE.html](https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html)

## 10. Repositorio

Codigo-fonte, README, testes e historico de desenvolvimento:

<https://github.com/davidsonsilva/desafio-dio-fraudes-cartao-credito>